



Machine Learning in Production

Process and Technical Debt

Assurance Cases: Example

Aurora's self-driving vehicles are acceptably safe to operate on public roads ^①

TOP LEVEL CLAIM



Aurora Safety Case

Breakout: Assurance Case for Recommender

As a group, tagging members, post to #lecture:

1. How would you argue that your recommendation system provides at least 99% availability?
2. What evidence would you provide *besides measurement of reliability over a certain time span*?



Process...

Fundamentals of Engineering AI-Enabled Systems

Holistic system view: AI and non-AI components, pipelines, stakeholders, environment interactions, feedback loops

Requirements:

System and model goals
User requirements
Environment assumptions
Quality beyond accuracy
Measurement
Risk analysis
Planning for mistakes

Architecture + design:

Modeling tradeoffs
Deployment architecture
Data science pipelines
Telemetry, monitoring
Anticipating evolution
Big data processing
Human-AI design

Quality assurance:

Model testing
Data quality
QA automation
Testing in production
Infrastructure quality
Debugging

Operations:

Continuous deployment
Contin. experimentation
Configuration mgmt.
Monitoring
Versioning
Big data
DevOps, MLOps

Teams and process: Data science vs software eng. workflows, interdisciplinary teams, collaboration points, technical debt

Responsible AI Engineering

Provenance,
versioning,
reproducibility

Safety

Security and
privacy

Fairness

Interpretability
and explainability

Transparency
and trust

Ethics, governance, regulation, compliance, organizational culture

Readings

Required Reading:

- Sculley, D, et al. “[Hidden technical debt in machine learning systems](#).” In Advances in neural information processing systems, pp. 2503–2511. 2015.

Suggested Readings:

- Fowler and Highsmith. [The Agile Manifesto](#)
- Steve McConnell. Software project survival guide. Chapter 3
- Kruchten, Philippe, Robert L. Nord, and Ipek Ozkaya. “[Technical debt: From metaphor to theory and practice](#).” IEEE Software 29, no. 6 (2012): 18-21.

Learning Goals

- Overview of common data science workflows (e.g., CRISP-DM)
 - Importance of iteration and experimentation
 - Role of computational notebooks in supporting data science workflows
- Overview of software engineering processes and lifecycles: costs and benefits of process, common process models, role of iteration and experimentation
- Contrasting data science and software engineering processes, goals and conflicts
- Integrating data science and software engineering workflows in process model for engineering AI-enabled systems with ML and non-ML components; contrasting different kinds of AI-enabled systems with data science trajectories
- Overview of technical debt as metaphor for process management; common sources of technical debt in AI-enabled systems

What is Process?

Software Process

“The set of activities and associated results that produce a software product”

A structured, systematic way of carrying out these activities

Q. Examples?

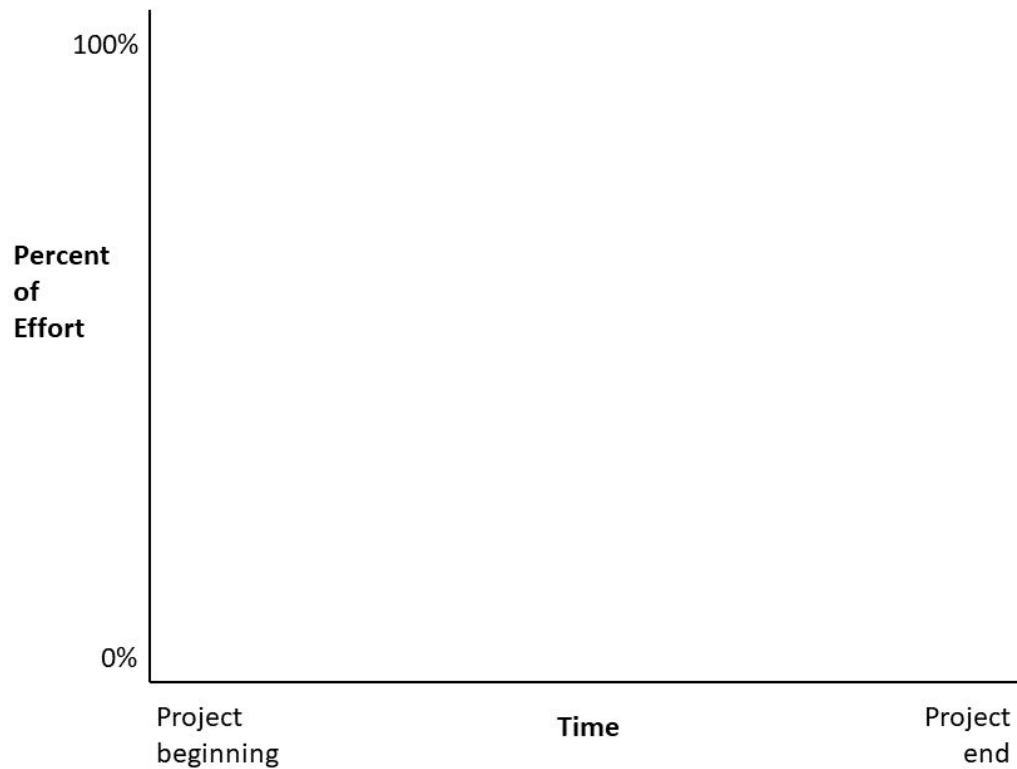
Developers dislike processes



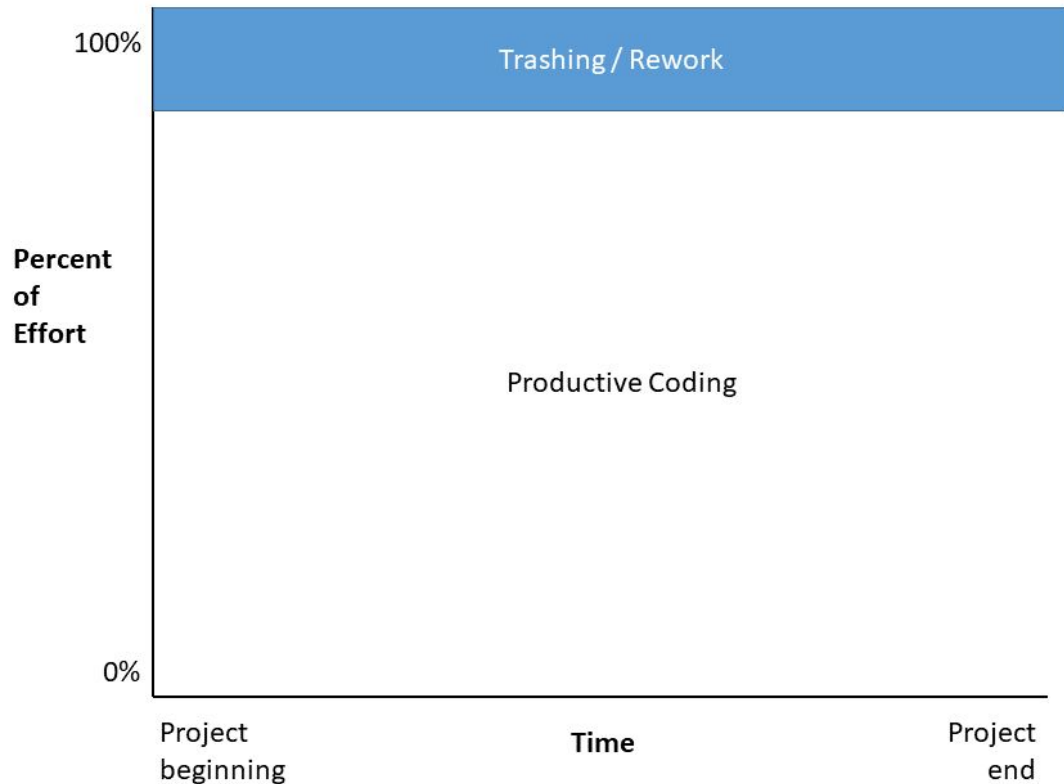
Follow
Process

Write
Code

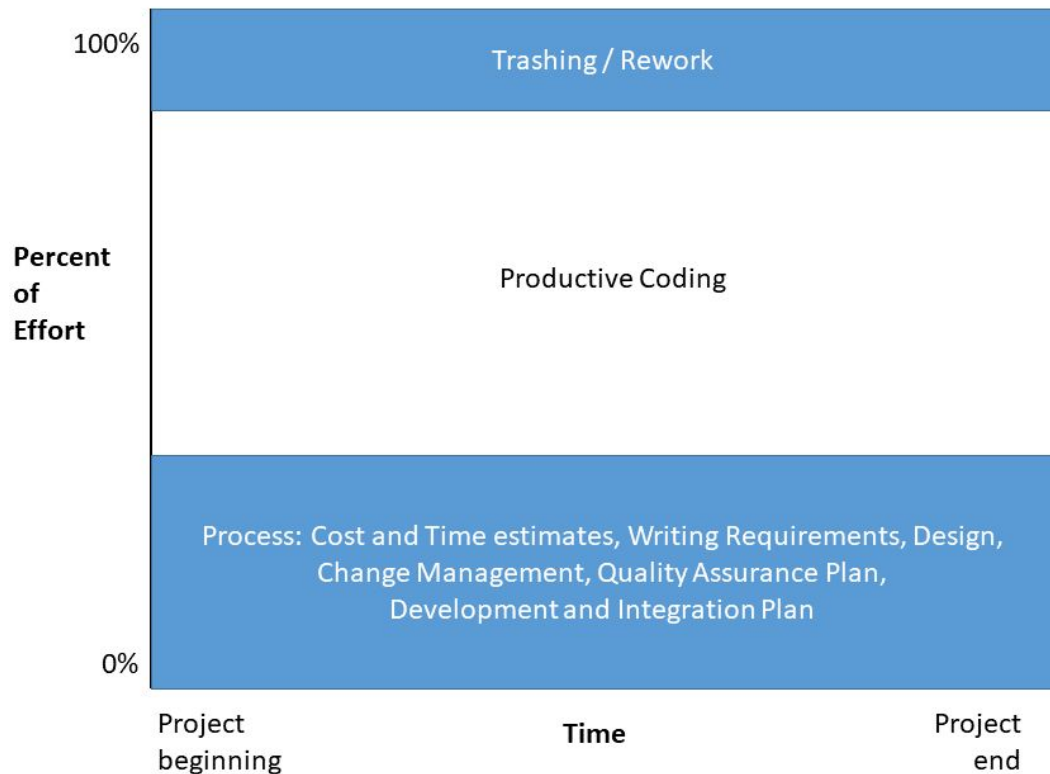
What developers want



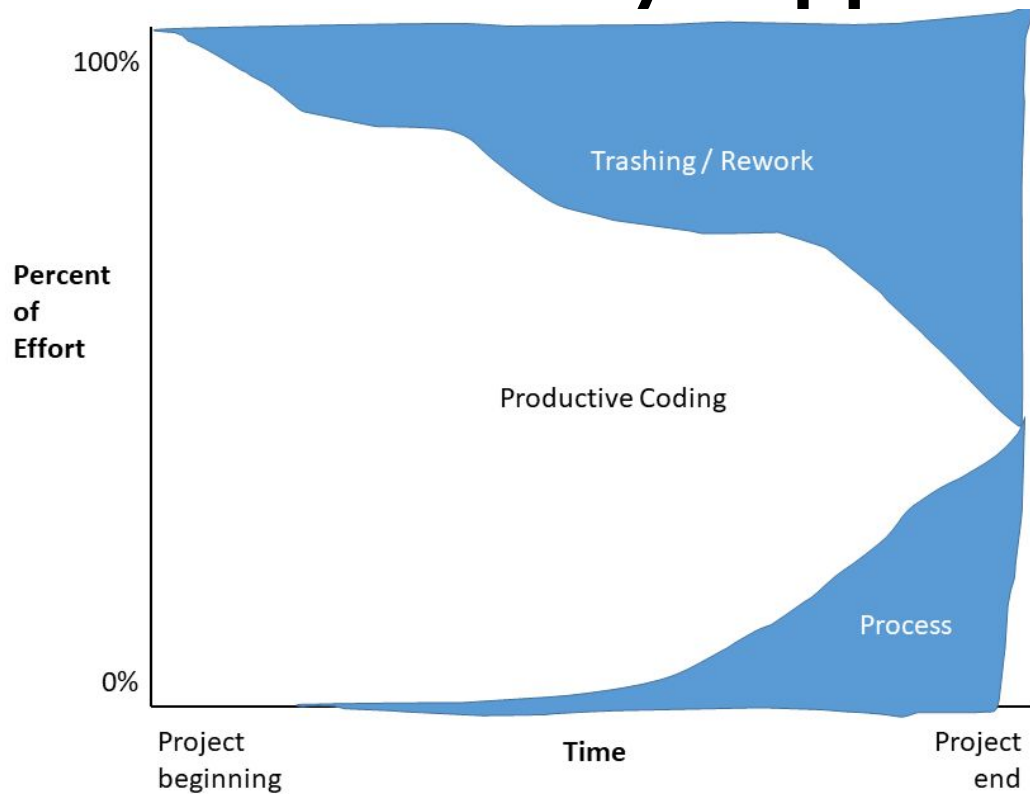
What developers want



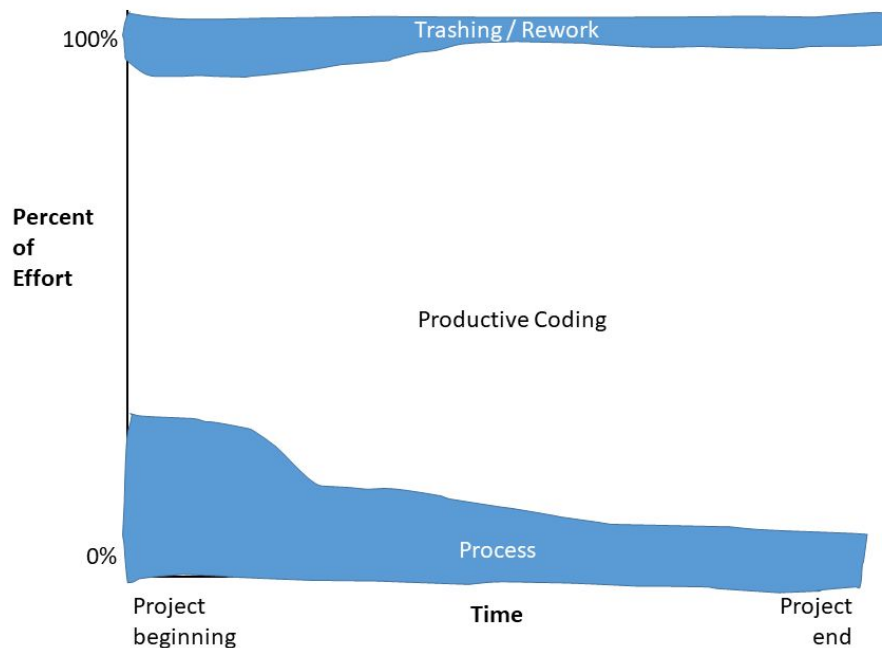
What developers think of processes



What eventually happens anyway



Hypothesis: Process increases flexibility and efficiency + Upfront investment for later greater returns



Survival Mode

Missed deadlines -> "solo development mode" to meet own deadlines

Ignore integration work

Stop interacting with testers, technical writers, managers, ...

→ Results in further project delays, added costs, poor product quality...

Examples of Process Problems

Mid-project informal agreement to changes suggested by customer or manager. Project scope expands 25-50%.

Late detection of requirements and design issues.

Bug reports collected informally, forgotten.

Integration of independently developed components at the very end of the project. Interfaces out of sync.

Accidentally overwritten changes, lost work.

When project is behind, developers are asked weekly for new estimates.

Time estimation:

II

*"Plans are nothing,
planning is everything"*

-Dwight D. Eisenhower



Data Science: Iteration and Exploration

Data Science is Iterative and Exploratory

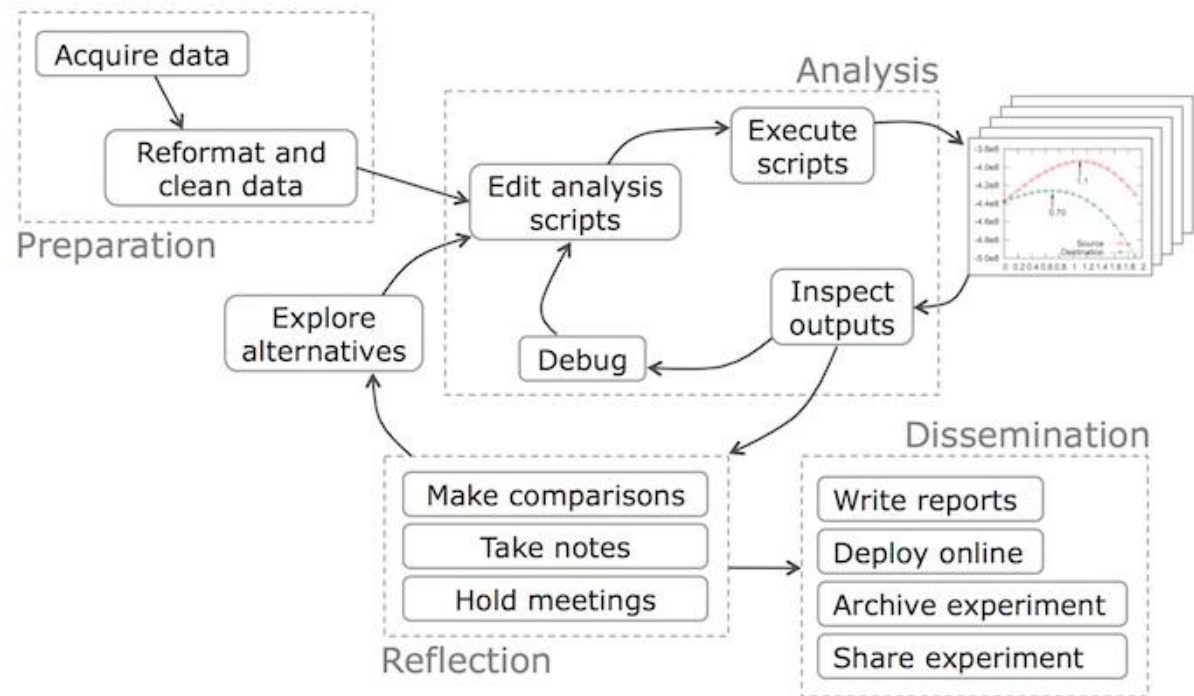
Science mindset: start with rough goal, no clear specification, unclear whether possible

Heuristics and experience to guide the process

Trial and error, refine iteratively, hypothesis testing

Go back to data collection and cleaning if needed, revise goals

Data Science is Iterative and Exploratory



Notebooks Support Iteration and Exploration

Quick feedback, similar to [REPL](#)

Visual feedback including figures and tables

Incremental computation: reexecuting individual cells

Quick and easy: copy paste, no abstraction needed, easy to share.

```
# load data collected from team1
import pandas as pd

url = 'http://128.2.25.78:8080/private/log1.clean'
df = pd.read_csv(url)
df.head()
```

	dayIdx	user	userAvgTime	location	dow	isWeekend	time
0	0	Pittsburgh66Correy	7.045001	Pittsburgh	6	True	0.000000
1	1	Pittsburgh66Correy	7.045001	Pittsburgh	7	True	6.883333
2	2	Pittsburgh66Correy	7.045001	Pittsburgh	1	False	6.816667
3	3	Pittsburgh66Correy	7.045001	Pittsburgh	2	False	7.383333
4	4	Pittsburgh66Correy	7.045001	Pittsburgh	3	False	0.000000

Data was preprocessed externally, identifying the time at a given day when the light was first turned on (12pm). Weather and sunrise information is not included here, though that'd be important. If the light was off this morning (quite common), 0 is recorded.

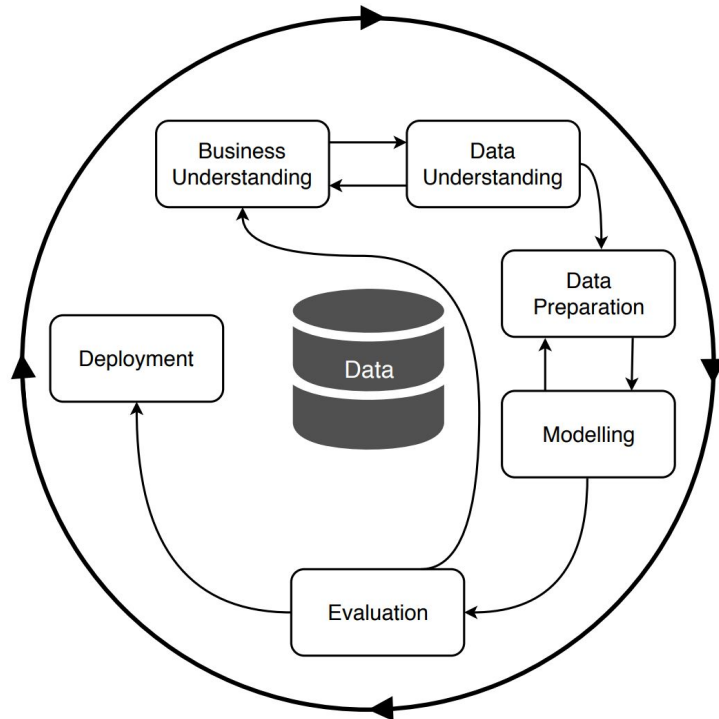
```
[ ] # just data encoding and splitting X and Y

X = df.drop(['time'], axis=1)
YnonZero = df['time'] > 0
Y = df['time']

from sklearn import preprocessing
# leDate = preprocessing.LabelEncoder()
# leDate.fit(X['date'])
# leDate.transform(X['date'])

X=X.apply(preprocessing.LabelEncoder().fit_transform)
X
```

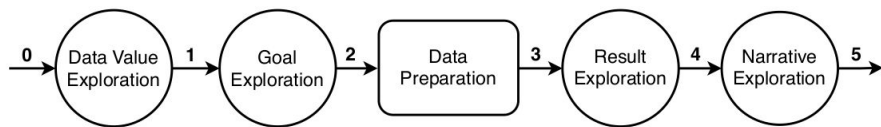
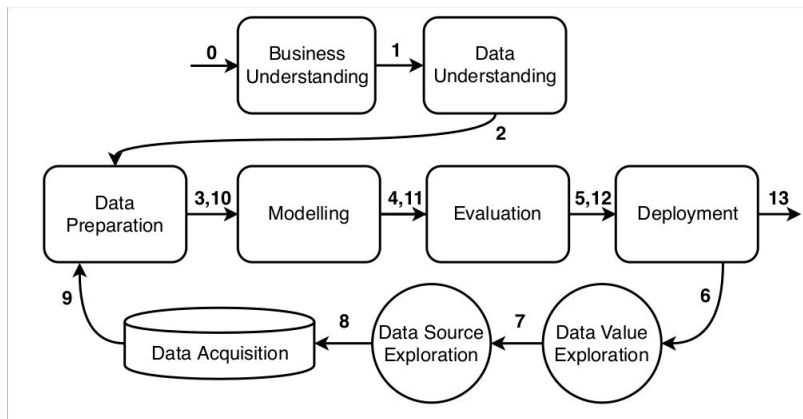
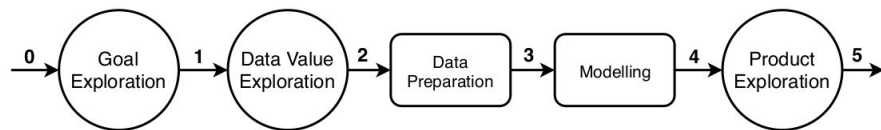
CRISP-DM: Cross-Industry Standard Process for Data Mining



Useful and widely-used for data science projects.

But: does not consider system or ongoing deployment.

Different Trajectories

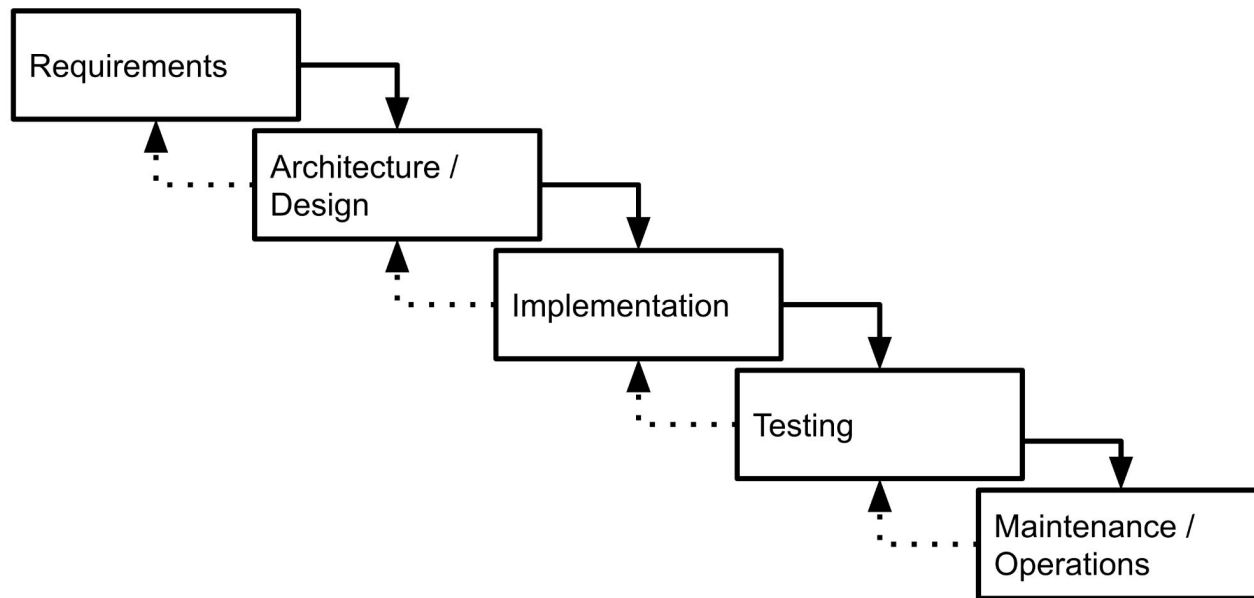


Software Process Models vs. ML/Data Science

Ad-hoc Processes

1. Discuss the software that needs to be written
2. Write some code
3. Test the code to identify the defects
4. Debug to find causes of defects
5. Fix the defects
6. If not done, return to step 1

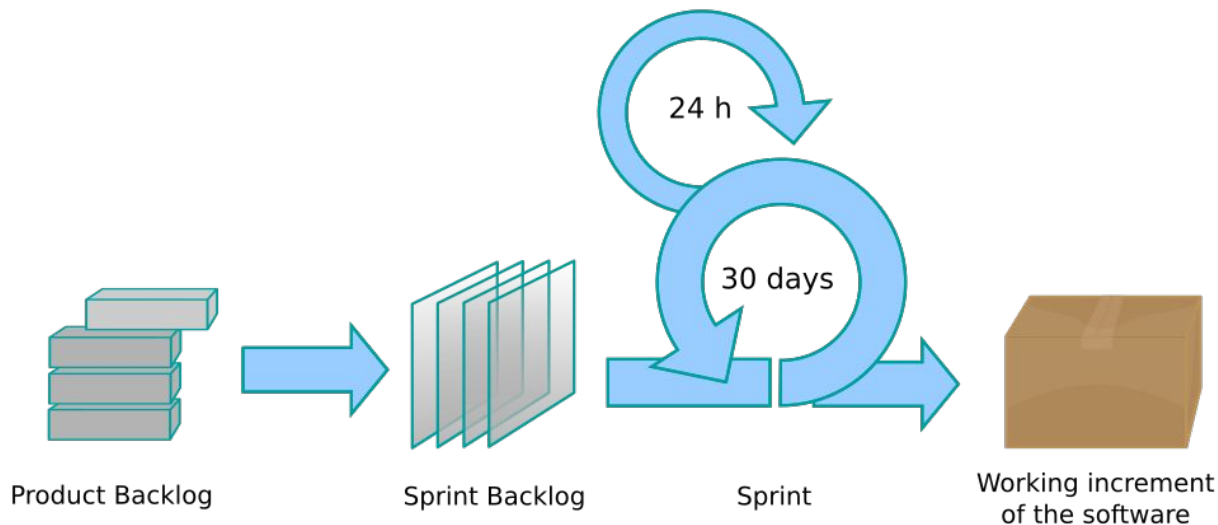
Waterfall Model



Understand requirements, plan & design before coding, test & deploy

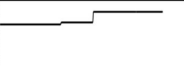
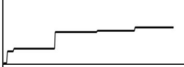
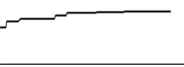
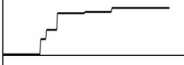
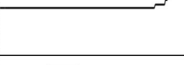
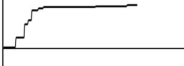
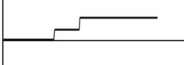
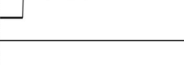
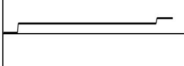
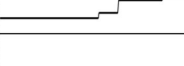
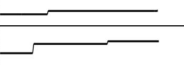
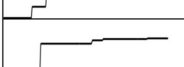
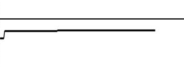
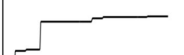
Machine Learning in Production • Christian Kaestner & Claire Le Goues, Carnegie Mellon University • Spring 2026

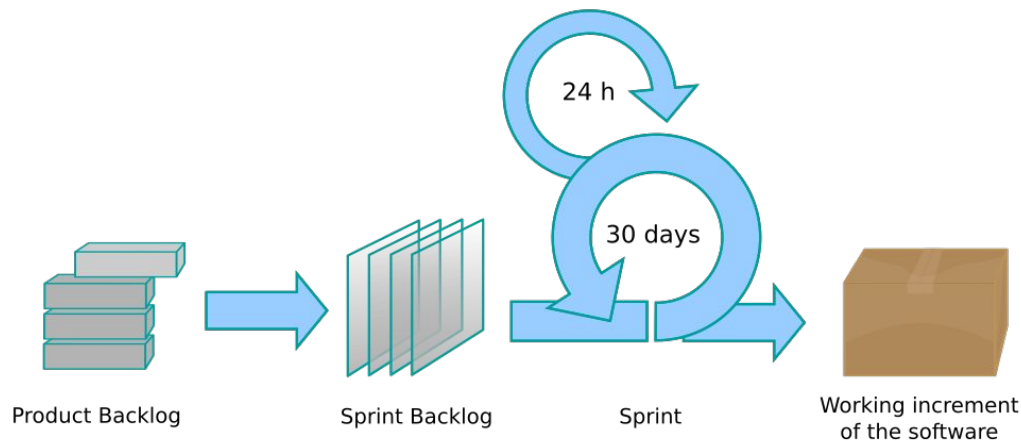
Constant Iteration: Agile



- Constant interactions with customers, constant replanning
- Scrum: Break into *sprints*; daily meetings, sprint reviews, planning

Discussion: Iteration in Notebook vs Agile?

	First 2 Hours	Second 2 Hours	Final Accuracy
TAP1			84.7%
TAP2	X	X	75.3%
TAP3			78.3%
TAP4			82.9%
TAP5			84.7%
TAP6			78.0%
TAP7			56.9%
TAP8			22.8%
TAP9			78.8%
TAP10			84.4%



Poor Software Engineering Practices in Notebooks?

```
# load data collected from team1
import pandas as pd

url = 'http://128.2.25.78:8080/private/log1.clean'
df = pd.read_csv(url)
df.head()
```

	dayIdx	user	userAvgTime	location	dow	isWeekend	time
0	0	Pittsburgh66Correy	7.045001	Pittsburgh	6	True	0.000000
1	1	Pittsburgh66Correy	7.045001	Pittsburgh	7	True	6.883333
2	2	Pittsburgh66Correy	7.045001	Pittsburgh	1	False	6.816667
3	3	Pittsburgh66Correy	7.045001	Pittsburgh	2	False	7.383333
4	4	Pittsburgh66Correy	7.045001	Pittsburgh	3	False	0.000000

Data was preprocessed externally, identifying the time at a given day when the light was first turned on (12pm). Weather and sunrise information is not included here, though that'd be important. If the light was not turned on this morning (quite common), 0 is recorded.

```
[ ] # just data encoding and splitting X and Y

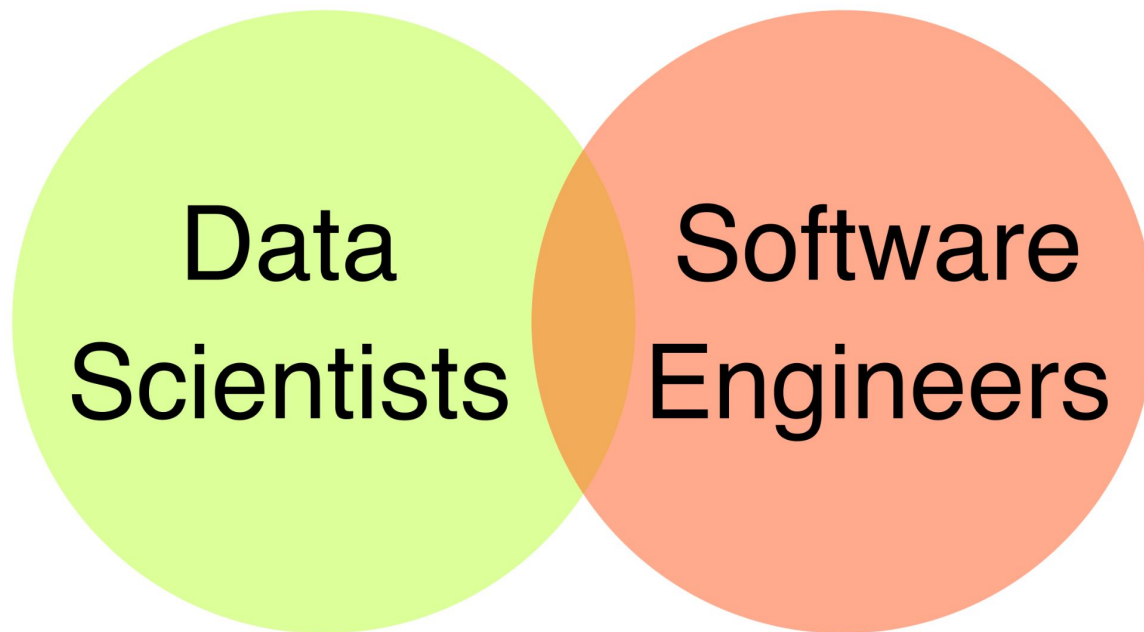
X = df.drop(['time'], axis=1)
YnonZero = df['time'] > 0
Y = df['time']

from sklearn import preprocessing
# leDate = preprocessing.LabelEncoder()
# leDate.fit(X['date'])
# leDate.transform(X['date'])

X=X.apply(preprocessing.LabelEncoder().fit_transform)
X
```

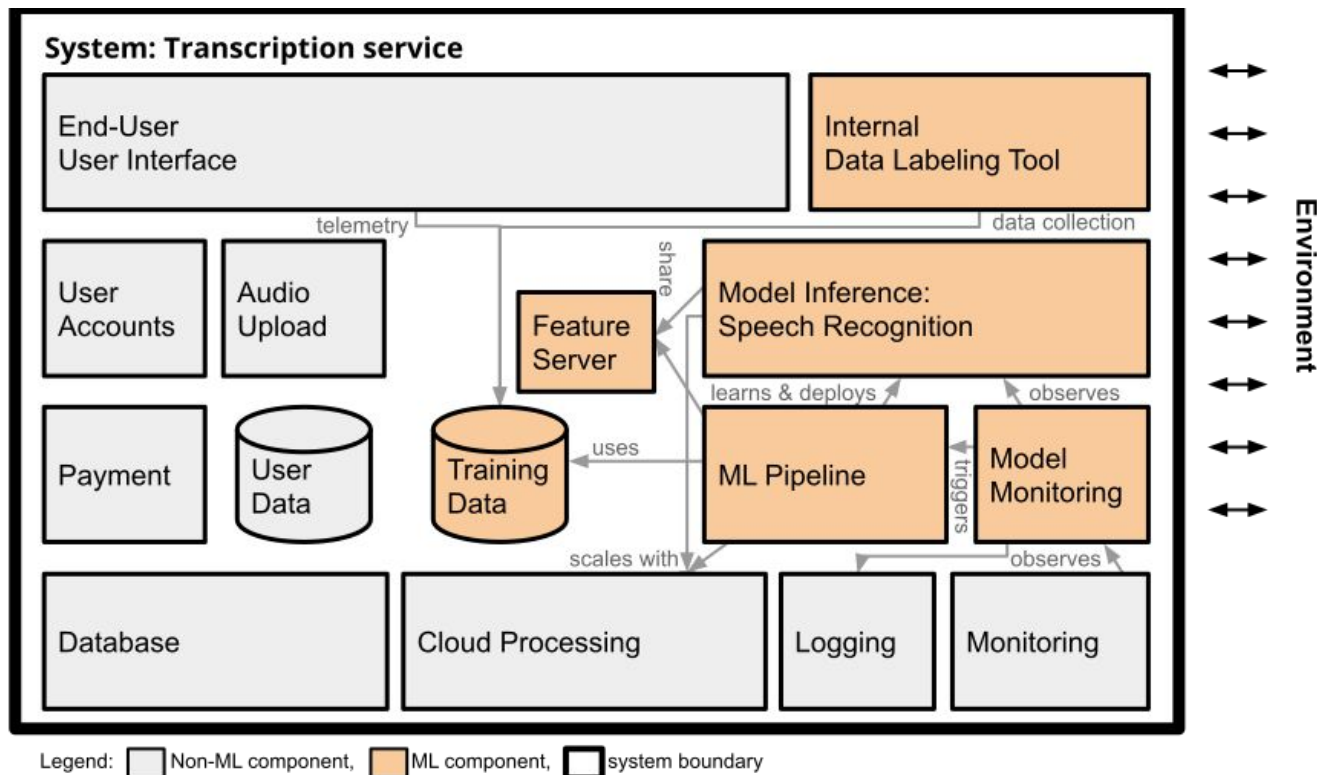
- Little abstraction
- Global state
- No testing
- Heavy copy and paste
- Little documentation
- Poor version control
- Out of order execution
- Poor development features (vs IDE)

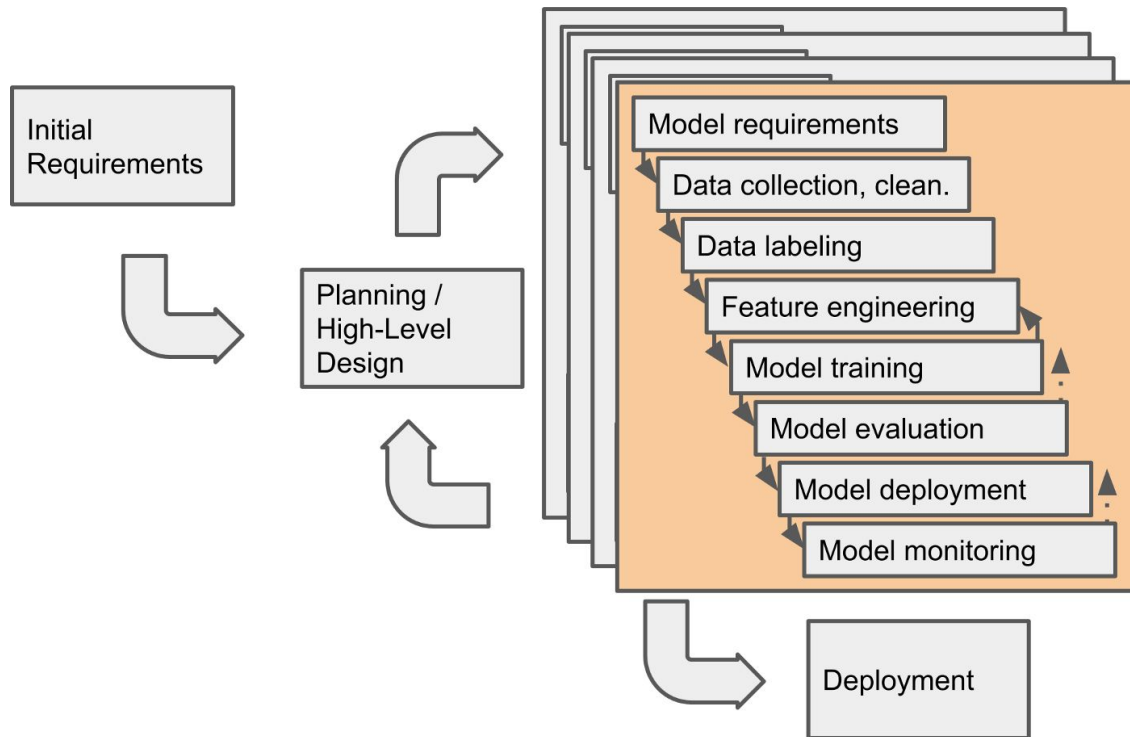
Note: exploration is fine; the problem is just transition to production.

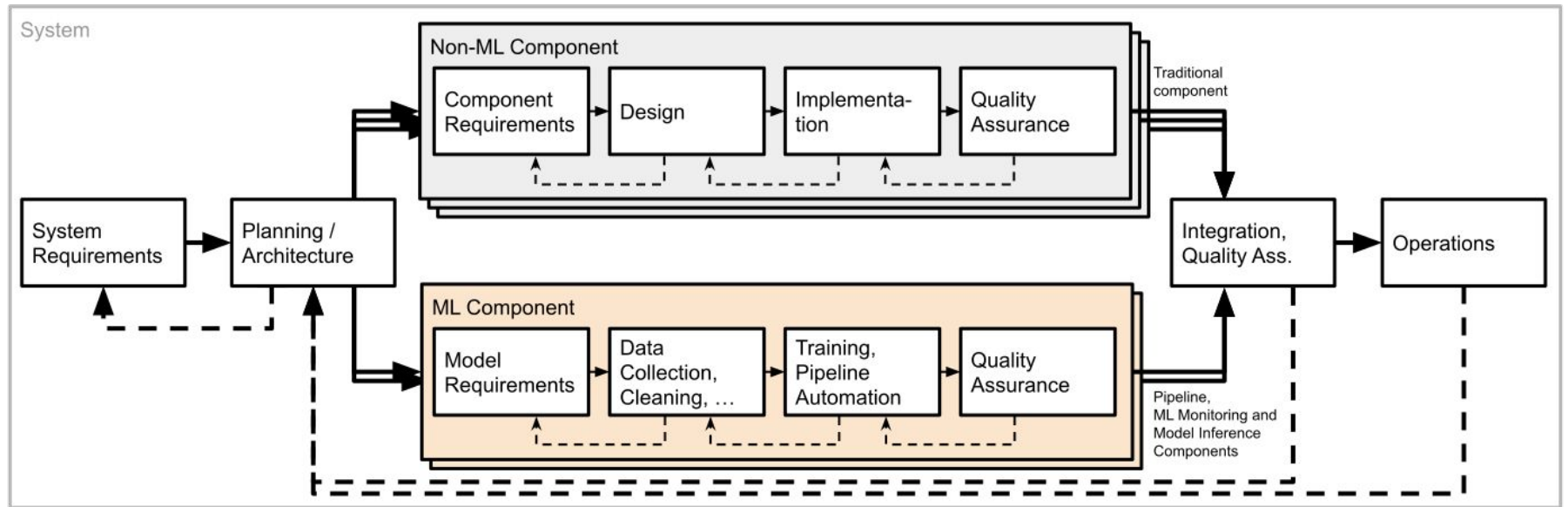


Integrated Process for AI-Enabled Systems

Recall: ML models are system components

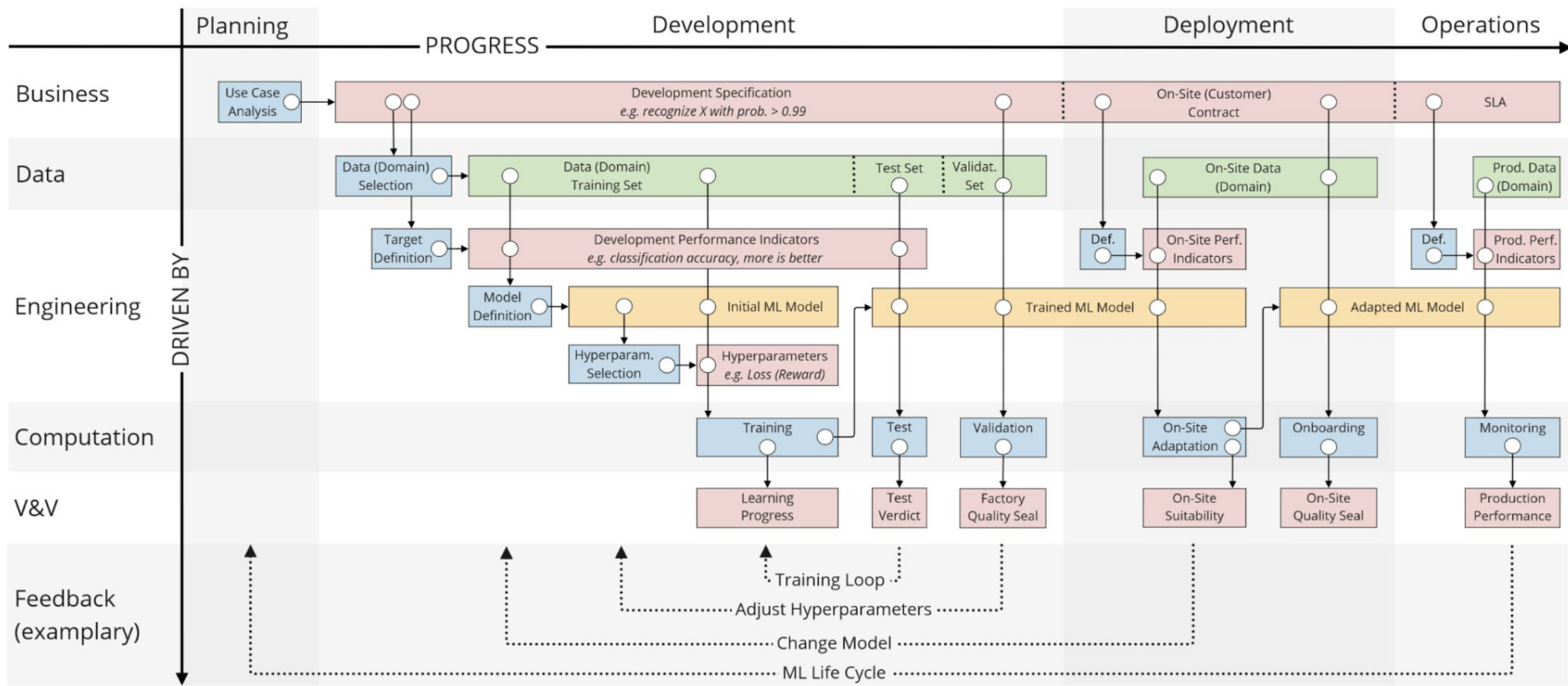




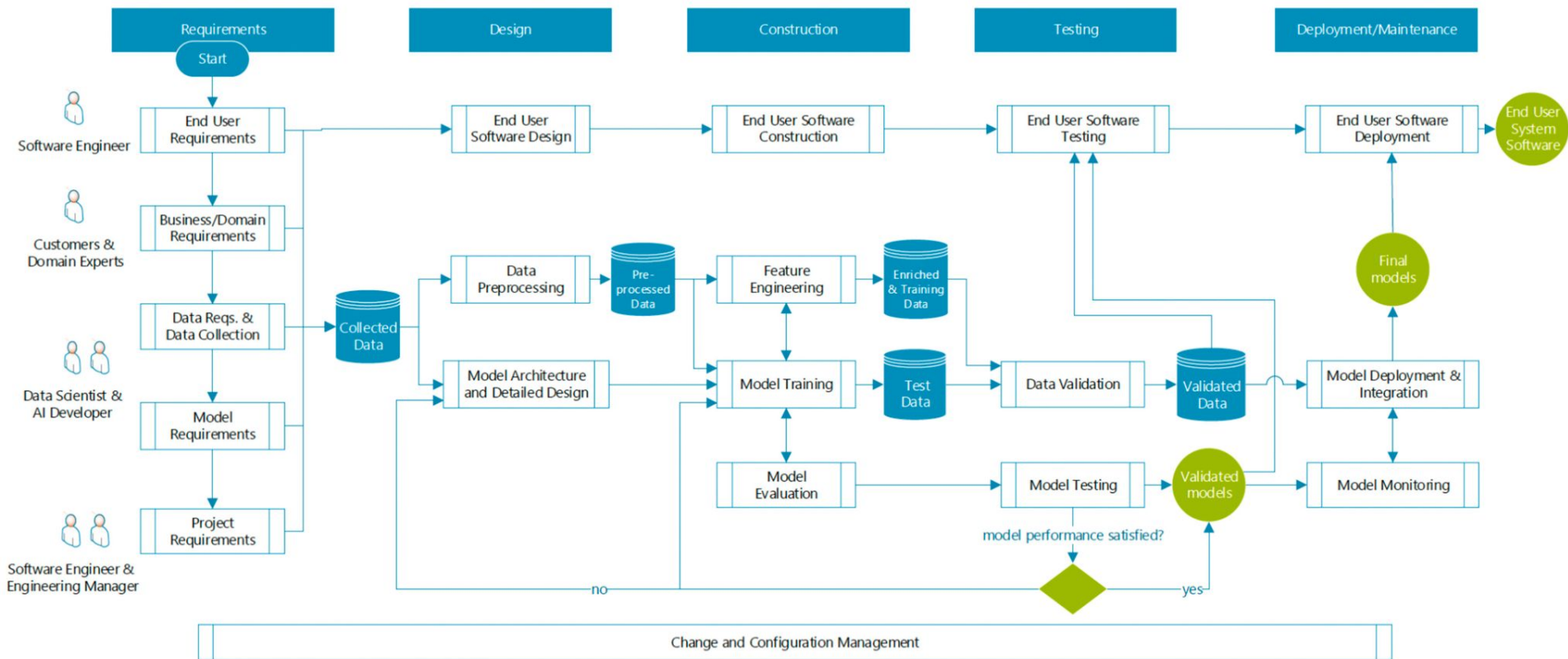


Process for AI-Enabled Systems

- Integrate Software Engineering and Data Science processes
- Establish system-level requirements (e.g., user needs, safety, fairness)
- Inform data science modeling with system requirements (e.g., privacy, fairness)
- Try risky parts first (most likely include ML components; ~spiral)
- Incrementally develop prototypes, incorporate user feedback (~agile)
- Provide flexibility to iterate and improve
- Design system with characteristics of AI component (e.g., UI design, safeguards)
- Plan for testing throughout the process and in production
- Manage project understanding both software engineering and data science workflows
- **No well-established "best practices" or workflow models**



Ritz, F., Phan, T., Sedlmeier, A., Altmann, P., Wieghardt, J., Schmid, R., ... & Gabor, T. (2022). Capturing dependencies within machine learning via a formal process model. In International Symposium on Leveraging Applications of Formal Methods (pp. 249-265). Cham: Springer Nature Switzerland.



Gurgen Erdogan, T., Altunel, H., & Tarhan, A. K. (2025). A Process Model for AI-Enabled Software Development: A Synthesis From Validation Studies in White Literature. *Journal of Software: Evolution and Process*, 37(1), e2743.

Trajectories

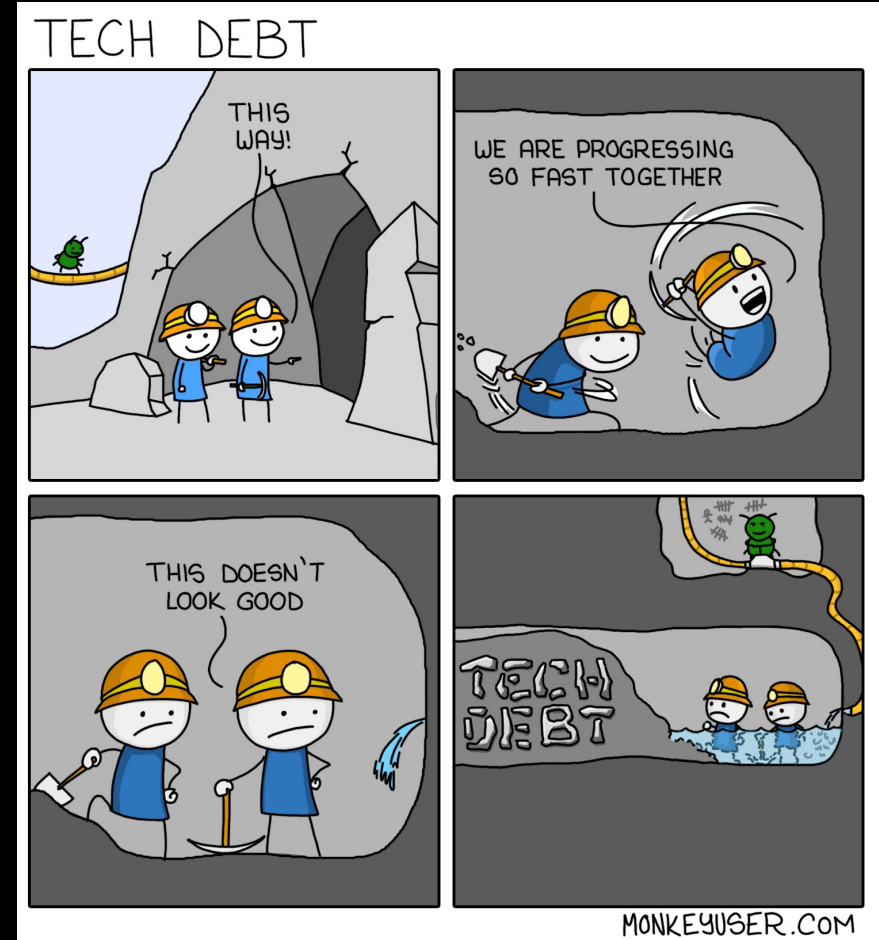
Not every project follows the same development process, e.g.

- Research only: Explore feasibility before thinking about a product
- **Small ML addition:** Product first, add ML feature later
- **Data science first:** Model as central component of potential product, build system around it

Different focus on system requirements, qualities, and upfront planning

Manage interdisciplinary teams and different expectations

Technical debt



Technical Debt Metaphor

Analogy to financial debt

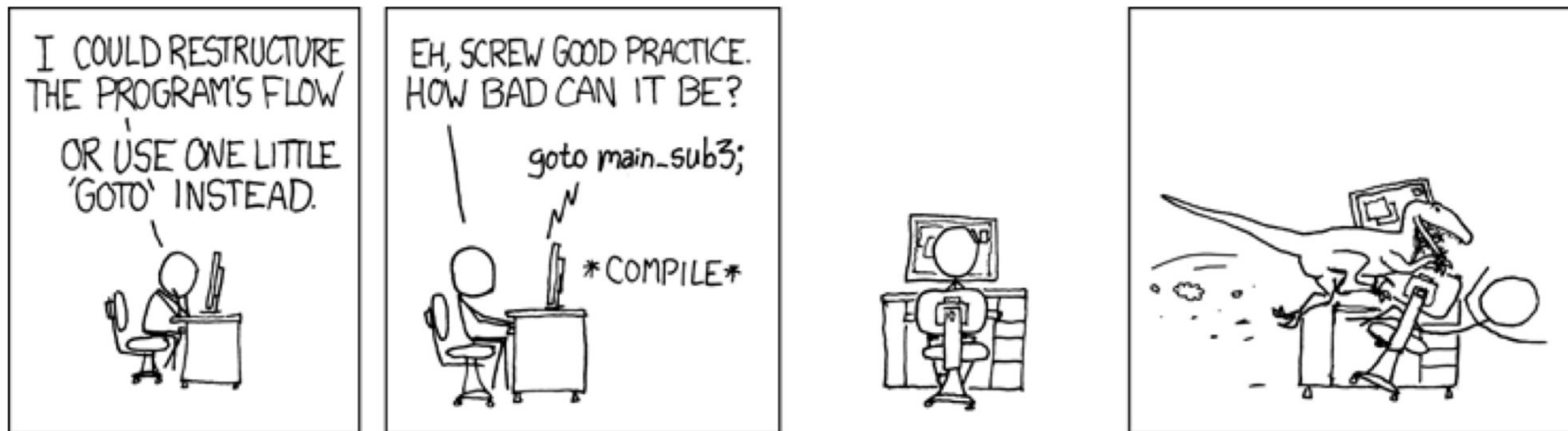
- Make a decision for an immediate benefit (e.g., release now)
- Accepting later cost (loss of productivity, higher maintenance and operating cost, rework)
- Debt accumulates and can suffocate project

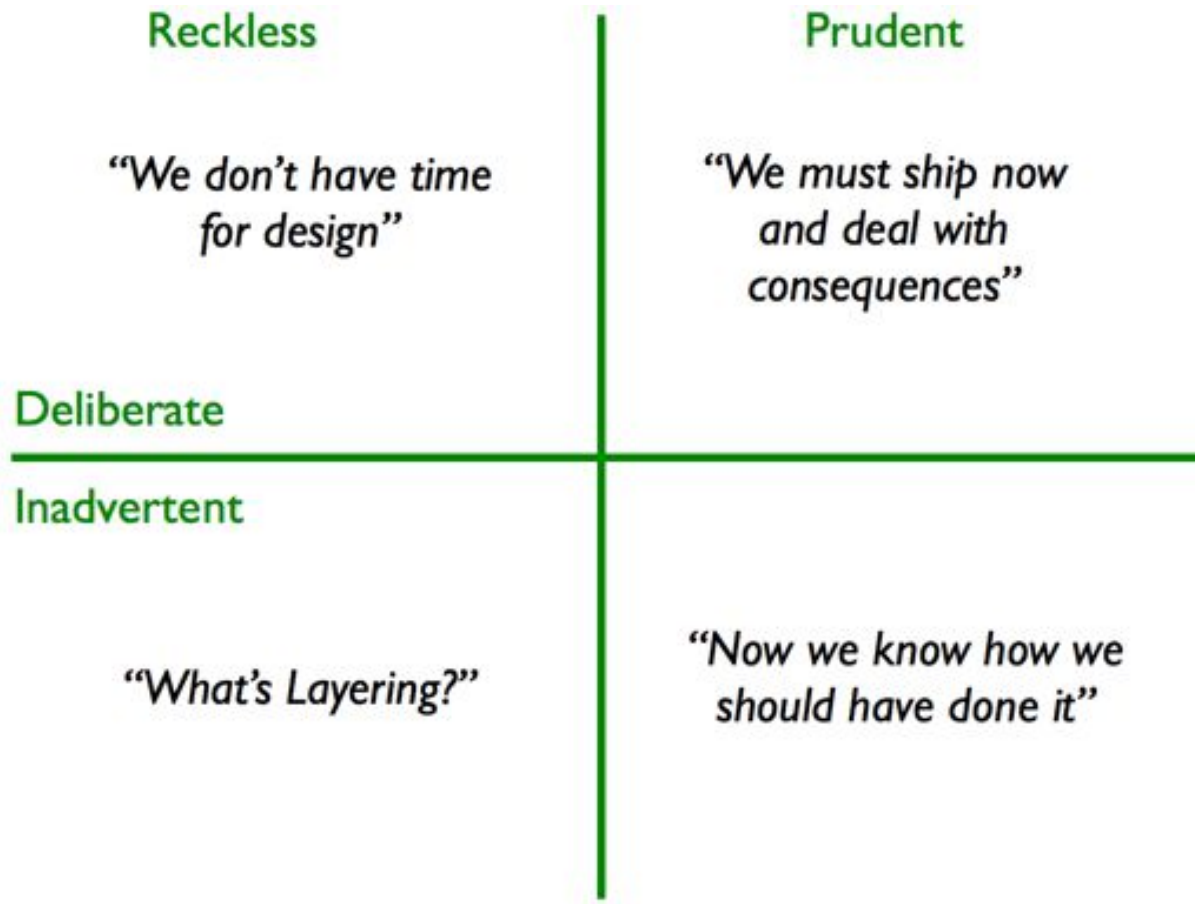
Ideally, a deliberate decision (short term tactical or long term strategic)

Ideally, track debt and plan for paying it down later

Q. Examples?

What causes technical debt?





Source: Martin Fowler 2009, <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>

Reckless

Prudent

*“We don’t have time
to plan for mistakes.”*

*“We must release now before
the competition and deal with
slow manual releases for now.”*

Deliberate

Inadvertent

“What’s data drift?”

*“We didn’t know about data
versioning and in retrospect it’s
good we didn’t waste a lot of
effort on it early on.”*

Technical Debt: Examples

Prudent & deliberate: Skip using a CI platform

- Reason for debt: Short deadline; test the product viability with alpha users using a prototype
- Debt payback: Refactoring effort to integrate the system into CI

Reckless & inadvertent: Forget to encrypt user credentials in DB

- Reason for debt: Lack of in-house security expertise
- Debt payback: Security vulnerabilities & fallout from an attack (loss of data); effort to retrofit security into the system

CACE: Changing Anything Changes Everything

(Sculley et al. call this “boundary erosion”; difficult to reason about components in isolation, little modularity)

Changing distribution of any feature can shift learned importance of all others.

Applies to hyperparameters, data selection, convergence thresholds...

Correction cascades compound the problem and create dependency chains.

Undeclared consumers create invisible entanglement.

ML Systems Anti-patterns (Sculley et al.)

Glue code: a mature ML system may be 5% model code and 95% supporting code to move data in and out of generic packages.

Pipeline jungles: data preparation evolves organically into a tangle of scrapes, joins, and intermediate files.

Dead experimental codepaths: old experiments left as conditional branches in production code.

- Knight Capital lost \$465M in 45 minutes partly due to obsolete codepaths.

Root cause: the paper argues for hybrid teams where engineers and researchers work together.

Example problems: ML and Technical Debt

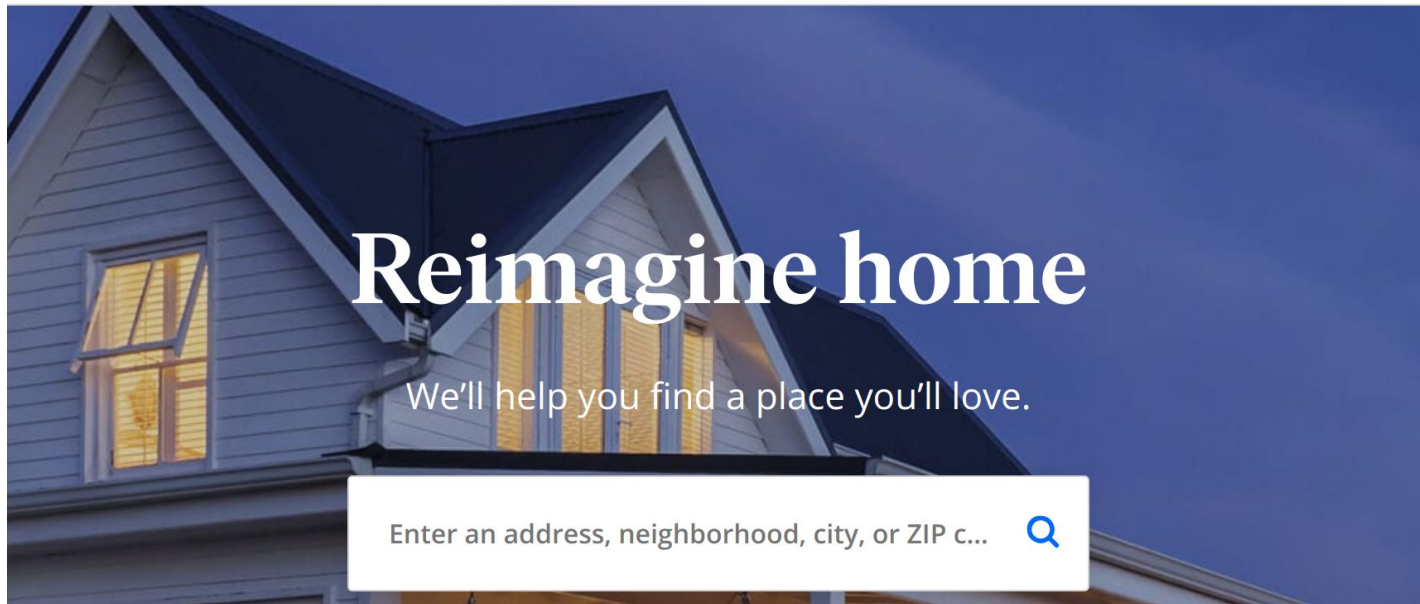
- Systems and models are tangled and changing one has cascading effects on the other
- Untested, brittle infrastructure; manual deployment
- Poor traceability
- Unstable data dependencies, replication crisis
- Data drift and feedback loops
- Magic constants and dead experimental code paths

Case Study: Real Estate Website

[Buy](#) [Rent](#) [Sell](#) [Home Loans](#) [Agent finder](#)



[Manage Rentals](#) [Advertise](#) [Help](#) [Sign in](#)



ML Component: Predicting Real Estate Value

Given a large database of house sales and statistical/demographic data from public records, predict the sales price of a house.

$$f(\textit{size}, \textit{rooms}, \textit{tax}, \textit{neighborhood}, \dots) \rightarrow \textit{price}$$



 Edit  Save  Share

3 bd | 1 ba | 2,090 Square Feet

541 S Graham St, Pittsburgh, PA 15232

● Off market

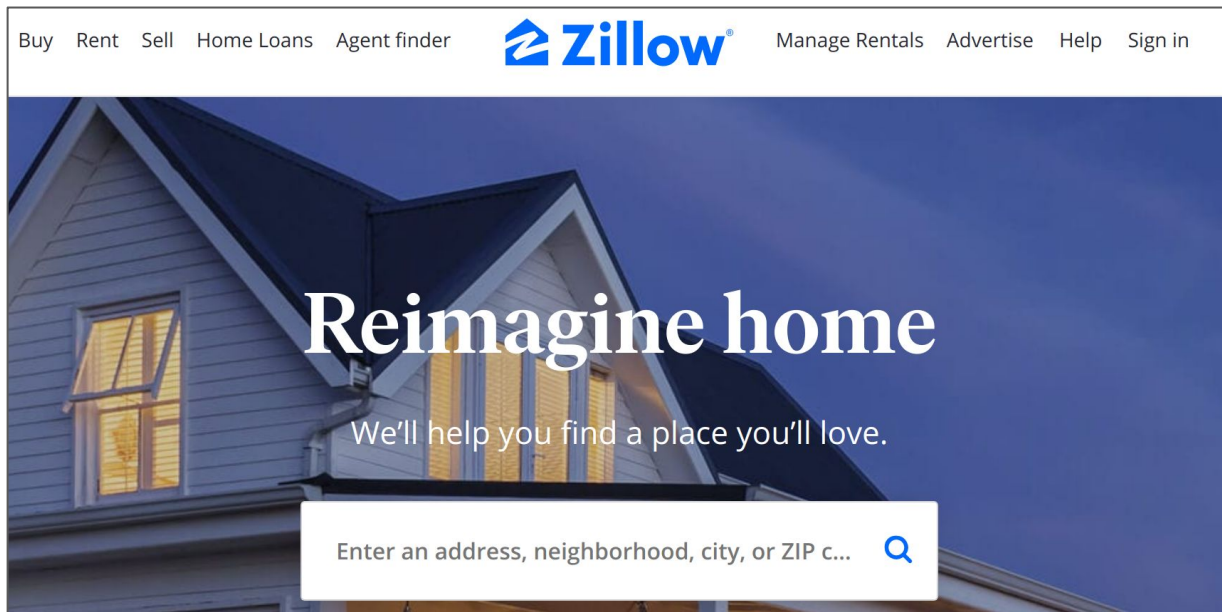
Zestimate®: \$384,287

Rent Zestimate®: \$2,195/mo

Est. refi payment: \$2,102/mo

 [Get current rates](#)

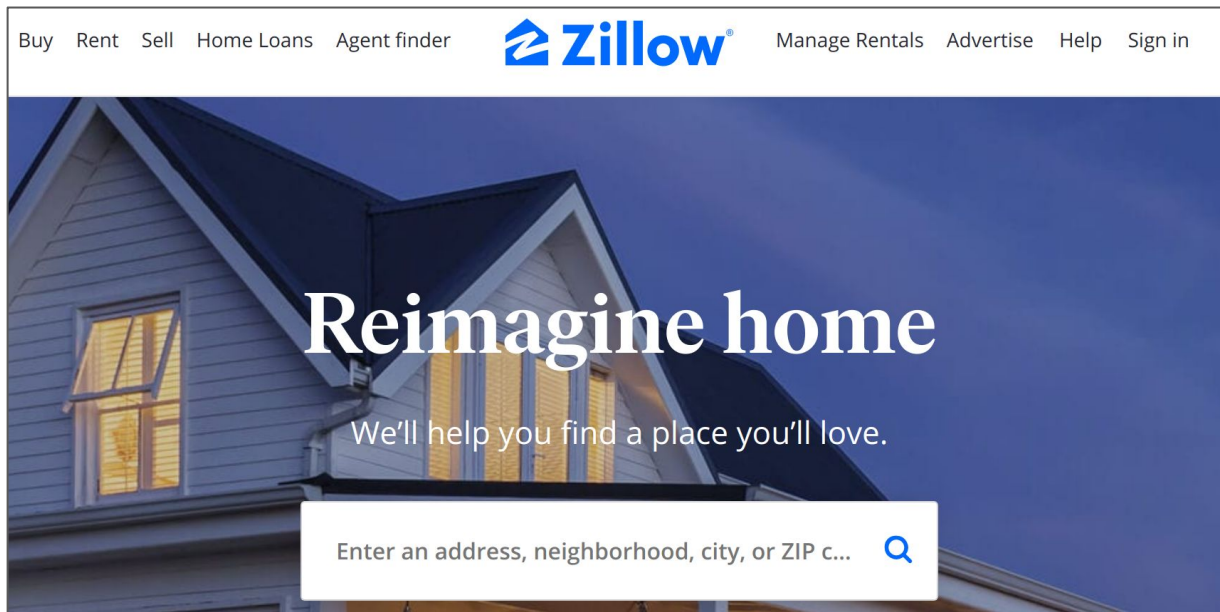
Breakout: Technical Debt from ML



As a group in #1 lecture, tagging members: Post two plausible examples technical debt in housing price prediction system:

1. Deliberate, prudent:
2. Reckless, inadvertent:

Breakout: Technical Debt from ML



Update your answer with examples from one or more of your group projects (M1 or M2).

Technical Debt through Notebooks?

Jupyter Notebooks are a gift from God to those who work with data. They allow us to do quick experiments with Julia, Python, R, and more --

[John Paul Ada](#)

```
# load data collected from team1
import pandas as pd

url = 'http://128.2.25.78:8080/private/log1.clean'
df = pd.read_csv(url)
df.head()
```

	dayIdx	user	userAvgTime	location	dow	isWeekend	time
0	0	Pittsburgh66Correy	7.045001	Pittsburgh	6	True	0.000000
1	1	Pittsburgh66Correy	7.045001	Pittsburgh	7	True	6.883333
2	2	Pittsburgh66Correy	7.045001	Pittsburgh	1	False	6.816667
3	3	Pittsburgh66Correy	7.045001	Pittsburgh	2	False	7.383333
4	4	Pittsburgh66Correy	7.045001	Pittsburgh	3	False	0.000000

Data was preprocessed externally, identifying the time at a given day when the light was first turned on (12pm). Weather and sunrise information is not included here, though that'd be important. If the light was on this morning (quite common), 0 is recorded.

```
[ ] # just data encoding and splitting X and Y

X = df.drop(['time'], axis=1)
YnonZero = df['time'] > 0
Y = df['time']

from sklearn import preprocessing
# leDate = preprocessing.LabelEncoder()
# leDate.fit(X['date'])
# leDate.transform(X['date'])

X=X.apply(preprocessing.LabelEncoder().fit_transform)
X
```

LLM and agent systems: slightly new problems, same old solutions!

Prompt sprawl: unversioned prompt templates accumulate.

Guardrail accumulation: safety filters, output validators, and retry logic can compose in a brittle way.

Hidden coupling through tool use: in agentic systems, changing one tool's behavior can cascade through the agent's reasoning.

Provider lock-in, opaque API dependencies: switching foundation model providers means total rework; provider can change or deprecate model at any time.

Controlling Technical Debt

Creating a tech debt management maturity model and accompanying assessment for teams:

- **Reactive** approach – no process
- **Proactive** approach – deliberately identify and track tech debt; make decisions about its urgency and importance relative to other work
- **Strategic** approach – designating specific champions
- **Structural** approach – embedding tech debt considerations into the developer workflow; standardizing how it is handled across a larger org



Defining, Measuring, and Managing Technical Debt

Ciera Jasan and Collin Green

WARD CUNNINGHAM INTRODUCED the metaphor underlying the term *technical debt* in a 1992 experience report, where he described how his company incrementally extended a piece of financial software:

Mature sections of the program have been revised or rewritten many times providing the consolidation that is key to understanding and continued incremental development. [...] Although immature code may work fine and be completely acceptable to the customer, excess quantities will make a program unmasterable, leading to extreme specialization of programmers and finally an inflexible product. Shipping first time code is like going into debt. A little debt speeds development so long as it is paid back promptly with a rewrite. [...] The danger occurs when the debt is not repaid. Every minute spent on not-quite-right code counts as interest on that debt. Entire engineering organizations can be brought to a stand-still under the

debt load of an unconsolidated implementation [...].¹

A few things stand out about this original use of the technical debt metaphor as resonant with a human-centered approach to developer productivity²:

- It invokes the properties of humans (for example, the ability to comprehend the product's code) as an important determinant of software engineering (SWE) process outcomes.
- It frames technical debt as arising mostly from nontechnical (business and organizational) factors.
- It focuses on the practical consequences of technical debt for engineering organizations rather than merely the consequences that exist in the code.

Cunningham's example of technical debt in that 1992 report is specific, but the metaphor is more general and centers on decision making and tradeoffs between the speed of delivery and the quality of the product. Over time, the technical debt metaphor has been used and misused in a wide variety of contexts

and to describe a wide variety of behaviors, processes, and SWE scenarios. (For example, Cunningham did not intend the technical debt metaphor to excuse engineers writing bad code.³) As a result, the term itself—*technical debt*—can be difficult to understand and interpret.

Since 2018, our quarterly engineering satisfaction survey has asked engineers to indicate the extent to which they are “hindered by unnecessary complexity and technical debt,” and the percentage of engineers who feel hindered is substantial. Early on, our engineering leads wanted to know more: What is the root cause of technical debt? How can we fix it? When engineers say technical debt is slowing them down, what do they even mean? Is “technical debt” just a catch-all term for anything an engineer dislikes? These questions motivated us to systematically investigate what technical debt means to engineers, how we might measure it, and how we might better manage technical debt. We wanted to deeply understand technical debt as *engineers chose to use the term*, rather than defining it for them, and to work on addressing technical debt from there.

ML and Technical Debt

Often reckless and inadvertent in inexperienced teams

ML can seem like an easy addition, but it may cause long-term costs

Needs to be maintained, evolved, and debugged

Goals may change, environment may change, some changes are subtle

Controlling Technical Debt from ML Components

- Avoid AI when not needed
- Understand and document requirements, design for mistakes
- Build reliable and maintainable pipelines, infrastructure, good engineering practices
- Test infrastructure, system testing, testing and monitoring in production
- Test and monitor data quality
- Understand and model data dependencies, feedback loops, ...
- Document design intent and system architecture
- Strong interdisciplinary teams with joint responsibilities
- Document and track technical debt
- ...

Summary

Data scientists and software engineers follow different processes

ML projects need to consider process needs of both

Iteration and upfront planning are both important, process models codify good practices

Deliberate technical debt can be good, too much debt can suffocate a project

Easy to amount (reckless) technical debt with machine learning

Further Reading

- Alex Serban, Koen van der Blom, Holger Hoos, and Joost Visser. 2020. "[Adoption and Effects of Software Engineering Best Practices in Machine Learning](#)." In Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (ESEM '20).
- Studer, Stefan, Thanh Binh Bui, Christian Drescher, Alexander Hanuschkin, Ludwig Winkler, Steven Peters, and Klaus-Robert Mueller. "[Towards CRISP-ML \(Q\): A Machine Learning Process Model with Quality Assurance Methodology](#)." arXiv preprint arXiv:2003.05155 (2020).
- Martínez-Plumed, Fernando, et al. "[CRISP-DM Twenty Years Later: From Data Mining Processes to Data Science Trajectories](#)." IEEE Transactions on Knowledge and Data Engineering (2019).
- Kaestner, Christian. [On the process for building software with ML components](#). Blog Post, 2020

Further Reading 2

- Patel, Kayur, James Fogarty, James A. Landay, and Beverly Harrison. "[Investigating statistical machine learning as a tool for software development](#)." In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 667-676. 2008.
- Yang, Qian, Jina Suh, Nan-Chen Chen, and Gonzalo Ramos. "[Grounding interactive machine learning tool design in how non-experts actually build models](#)." In *Proceedings of the 2018 Designing Interactive Systems Conference*, pp. 573-584. 2018.
- Fowler and Highsmith. [The Agile Manifesto](#)
- Steve McConnell. Software project survival guide. Chapter 3
- Pfleeger and Atlee. Software Engineering: Theory and Practice. Chapter 2
- Kruchten, Philippe, Robert L. Nord, and Ipek Ozkaya. "[Technical debt: From metaphor to theory and practice](#)." IEEE Software 29, no. 6 (2012): 18-21.